Analysis and Implementation of a Real-Time Demonstrator on the RTIC Runtime

Gianluca Bresolin
gianluca.bresolin@studenti.unipd.it
Università degli Studi di Padova
Padova, Veneto, Italy

Tommaso Prandin
tommaso.prandin@studenti.unipd.it
Università degli Studi di Padova
Padova, Veneto, Italy

Abstract

This paper presents the implementation and analysis of a real-time demonstrator originally developed in *Ada* using the *Ravenscar Profile*, re-implemented in *Rust* using the *RTIC* framework. The demonstrator is designed to run on an *STM32F4* microcontroller and uses a *Fixed-Priority Scheduling* policy combined with *Priority Ceiling Protocol (PCR)* for resource sharing. The analysis is performed using the *MAST* tool to verify timing properties and analyze system performance under various workloads. The results demonstrate the feasibility of adopting *Rust* and *RTIC* for real-time applications, while also highlighting differences and considerations compared to the original *Ada* implementation.

CCS Concepts

• **Software and its engineering → Real-time systems software**; **Embedded software**; *Software architectures.*

Keywords

Real-Time Kernels, Real-Time Systems, Embedded Software, Software Architectures

ACM Reference Format:
Gianluca Bresolin and Tommaso Prandin. 2025. Analysis and Implementation of a Real-Time Demonstrator on the RTIC Runtime. In *Proceedings of Real-Time Kernels and Systems Exam (RTKS '25).* ACM, New York, NY, USA, 10 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Real-time embedded systems demand precise control over timing, concurrency and resource management. Such systems are often used in critical applications where failure to meet timing constraints can lead to severe consequences. Hence, developing a real-time system requires careful consideration of the underlying runtime environment, necessitating a clear understanding of how the system will behave in order to provide sufficient guarantees on meeting timing requirements.
The choice of the runtime environment is a crucial aspect in the design of a real-time system, as it can significantly impact the system's ability to meet its desired behaviour through reliable mechanisms it offers. One of the most popular languages that meets the needs of real-time systems is *Ada*, which has been widely used in the development of safety-critical systems due to its deterministic constructs that allow high analysability of the code. While *Ada* remains a robust choice, recent advances in the *Rust* programming language have introduced new opportunities for real-time systems development.
In this paper, a real-time system originally implemented using the *Ada Ravenscar Profile* is re-implemented in *Rust* using the*Real-Time Intrupput-driven Concurrency (RTIC)* framework [3], which is responsible for managing the scheduling of tasks through a *Fixed-Priority Scheduling* and the resource sharing in the system under *Stack Resource Policy (SRP).*
For the analysis of the system, the *MAST* tool is used to verify the timing properties and to analyze the system under different workloads, which is run on a *STM32F4* microcontroller.
The goal of this work is to demonstrate the feasibility of using Rust and RTIC for real-time systems, while also providing some considerations and comparison with the original *Ada* implementation.
The rest of the paper is organized as follows. Section 2 presents a formal description of the system model. Section 3 introduces the *Rust Runtime*, in particular the *RTIC* framework and its key concepts. Sections 4 and 5 discuss the measurement of execution times and the detection of deadline misses, respectively. Section 6 provides an overview of the *MAST* tool and its analysis capabilities. In Section 7, we present the *Fixed-Priority Scheduling (FPS)* analysis of the system. Finally, Section 8 concludes the paper reporting the main findings.

2 System Model

The software architecture follows the one proposed in "*Guide for the Use of Ada Ravenscar Profile in High Integrity Systems*" [10]. It is comprised by one periodic producer task that can activate two other sporadic tasks. Additionally, there is a sporadic event server that manages the occurrence of external events. In the implemented application events are generated by a periodic task that emulate user interaction, for repeatability reasons.

Table [1] contains the list of tasks constituting the application, together with their typology, deadline, period (if applicable) and priority. Priorities have been assigned using the *Deadline Monotonic* algorithm, which assigns priorities in decreasing order, starting with jobs of smaller relative deadline. It is an optimal algorithm for constrained deadlines and fixed-priority scheduling [14, p. 119].
The deadline watchdogs have been given the highest priority in order to detect overrun without interference from other tasks (see section 5 for furhter details).

RTKS '25, Padova, Italy

ACM ISBN 978-1-4503-XXXX-X/2018/06
https://doi.org/XXXXXXX.XXXXXXX

Task	Task Type	Period (ms)	Deadline (ms)	Priority
Activation Log Reader	Sporadic	3000	1000	3
On Call Producer	Sporadic	5000	800	5
Regular Producer	Periodic	1000	500	7
External Event Server	Sporadic (interrupt)	5000	100	11
Interrupt Generator	Periodic	5000	50	13
ALR Deadline Watchdog	Sporadic	3000	1000	15
OCP Deadline Watchdog	Sporadic	5000	800	15
RP Deadline Watchdog	Periodic	1000	500	15
EES Deadline Watchdog	Sporadic (interrupt)	5000	100	15

Table 1: List of tasks in the system, with their parameters

Resource	Users	Ceiling π_R
Activation Log	Activation Log Reader, External Event Server	11
Event Queue	External Event Server, External Event ISR	11
Request Buffer	Regular Producer, On-Call Producer	7
ALR Deadline Object	Activation Log Reader, ALR Deadline Watchdog	15
OCP Deadline Object	On-Call Producer, OCP Deadline Watchdog	15
RP Deadline Object	Regular Producer, RP Deadline Watchdog	15
EES Deadline Object	External Event Server, EES Deadline Watchdog	15

Table 2: List of shared resources in the system, with their users and priority ceilings

The tasks in the system share various resources. Access is regulated by variants of the *Priority Ceiling Protocol* [16] depending on the implementation: the Ada system implements the *Immediate Priority Ceiling Protocol* [11], while the Rust system uses *Stack Resource Policy* [9].
While the implementation is slightly different, both protocols show the same properties and response-time behavior. They require to specify the ceiling value for each shared resource, which is obtained by picking the maximum priority value among all the task using the given resource.

Table [2] contains the list of all the shared resources in the system, together with their users (tasks) and their priority ceiling.

In order to emulate a realistic workload for task requiring some kind of computation, the small variant of the Whetstone benchmark [12] is used. It includes different kind of computations: integer, floating-point, trigonometry, branches and function calls, with the aim of testing the system under a wide range of loads.
This workload is a parameter that we used in the analysis section to create different workload levels and observe the resulting behavior of the system, first by simulating it using the MAST tool [2], then by running the application in a real setting.

To this end, we have used the *Olimex STM32H405* [15] development board, together with a *STLinkv2* [4] debugging probe.

2.1 Tasks Description

2.1.1 Activation Log Reader. The *Activation Log Reader* is a sporadic task which awaits on a signal received from the *Regular Producer*, and then performs its internal operation which consists of an artificial Whetstone workload, followed by a reading of the status of the *Activation Log* and a print that notifies the end of its sporadic activation.

2.1.2 On-Call Producer. The *On-Call Producer* is a sporadic task that waits for a request deposit into the *Request Buffer*, then reads it and runs a Whetstone workload with load equal to the extracted value. The requests are sent by the *Regular Producer*, and they are meant to represent a scenario in which a main task delegates some work to a helper task in the event of an overload.

2.1.3 Regular Producer. The *Regular Producer* is a periodic task that runs a Whetstone workload with a predefined load value, then, based on a predetermined criterion, can release the *Activation Log Reader* and send a work request to the *On-Call Producer*.

2.1.4 External Event Server. The *External Event Server* is a sporadic task that waits for events put onto the *Event Queue* by the interrupt handler bound to the simulated external event. It updates the *Activation Log* counter on each activation.

2.1.5 Deadline Watchdogs. For every task in the system there is a watchdog monitor task that checks for deadline overruns. It has the maximum priority to prevent interference. It maintains the number of deadline misses and whenever it detects one it prints a warning message (see section 5 for further details).

Figure [1] shows a high-level view of the system architecture

3 Rust Runtime

Starting from the demo application provided by the Ada guide [10], we then went on implementing an equivalent system fully written in Rust.

Compared to Ada, Rust does not provide a reference runtime implementation, leaving to third-party developers this incumbency. This is advantageous from a flexibility and ecosystem development standpoint, since it encourages the growth of a competitive runtime

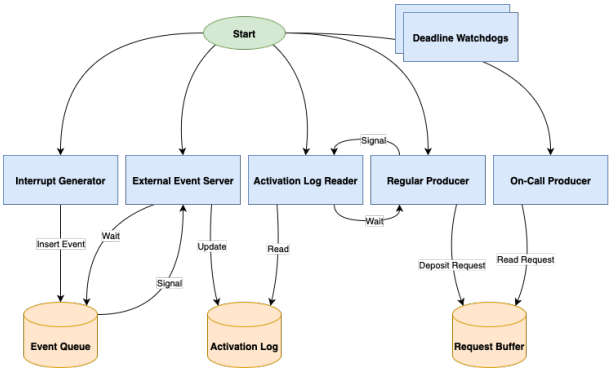

Figure 1: System architecture diagram

ecosystem, and fosters the development of multiple different implementation, tailored to more specific use-cases. On the other hand, this clearly implies more ecosystem fragmentation, and could also result in lower "quality" of the runtimes, since a single reference implementation will naturally tend to concentrate resources for its development from all the actors interested in its usage or analysis, instead of dispersing them across multiple projects.

Thus, when designing and implementing a real-time system in Rust, some effort must be spent in order to identify and integrate the most appropriate runtime components for the system, having care to select compatible components that can work in harmony together. In our case this means choosing a *concurrency runtime* that provides the task abstraction and scheduling primitives, a *logging framework* and a *Hardware Abstraction Layer* that provides higher-level abstractions over the system hardware.

3.1 RTIC

As for the *concurrency runtime*, we have decided to use the *RTIC* framework [5]. It is generally considered to be a "concurrency framework", since it does not contain a full-fledged software kernel and relies on external *Hardware Abstraction Layer* to access peripherals.

RTIC implements the *Stack Resource Policy* for shared resource access management [9], with *Fixed-Priority Scheduling* [14, Chapter 6].

SRP has some really useful runtime properties for real-time systems, both for performance and especially for amenability to response-time analysis. Some of those properties are:

- Preemptive deadlock and race-free scheduling
- Tasks execute on a single shared stack
- Tasks run-to-completion with wait free access to shared resources
- Bounded priority inversion by a single critical section

Those come with the following set of requirements:

- Tasks are associated with static *pre-emption levels*
- Tasks execute on a single core
- Tasks do not voluntarily suspend (run-to-completion)
- Resource must be claimed and released in LIFO order

The motives behind these requirements and the proof of the properties are outside the scope of this work; they can be found in the original paper [9].

SRP based scheduling requires the set of static priority tasks and their access to shared resources to be known in order to compute a static ceiling π for each resource. The static resource ceiling $\pi(r)$ reflects the maximum static priority of any task that accesses the resource r. This is the reason why, despite its excellent properties, is not used in general-purpose OS: it requires the resource dependency graph to be known statically, which is generally not possible.

FPS is a very simple scheduling scheme: every task is statically given a fixed priority value (values must have an order), and all jobs released will inherit the priority of their task. At every point in time the running job is the one with the highest priority among all the *ready* ones.

The key idea of *RTIC* is to use the native features provided by the interrupt engine of ARM microcontrollers (*NVIC* [8]) to implement these two protocols, obtaining a *hardware-accelerated runtime* suitable for real-time systems, with minimal overhead.

In the case of the ARM Cortex-M architecture, each interrupt vector entry v_i is associated an *Interrupt Service Routine* isr_i, a static priority (p_i), an enabled- (en_i) and a pending-bit ($pend_i$).

An interrupt i is scheduled (run) by the hardware only if both of the following conditions are met:

(1) i is pending and enabled and has a priority higher than the `BASEPRI` register;
(2) i has the highest priority among interrupts meeting 1.

The *SRP* model for single-core static scheduling, on the other hand, states that a task should be scheduled (run) under the conditions:

(1) It is requested to run and has a static priority higher than the current system ceiling (Π);
(2) It has the highest static priority among tasks meeting 1.

The system ceiling Π is computed as the maximum priority ceiling of the currently held resources.

This shows a very close relationship between the scheduling logic implemented in hardware by the NVIC, and the model required by *SRP*. The mapping between logical tasks and actual hardware is quite straightforward:

- Each task t is mapped to an interrupt vector index i with the corresponding ISR $isr_i = t$ and given the correct static priority $p_i = p(t)$.
- The system priority ceiling Π is mapped to the `BASEPRI` register, which is updated at the entry and exit of every *critical section* (i.e. lock).

This mapping is computed at compile-time by *RTIC* (even priority ceilings of resources), thus is has no impact on the performance at runtime (*zero-cost abstraction*).

Actually, *RTIC* maps each task priority level to an ISR at the corresponding priority. Each ISR runs an *asynchronous executor* that handles tasks wrapped as *async* functions assigned to that priority level. This allows for *async/await* style programming in tasks, lifting the run-to-completion requirement for them, since each section of code between two yield points can be seen as an individual task. The compiler will reject any attempt to await while holding a resource (not doing so would break the strict LIFO requirement on resource usage under SRP) [5]. This in turn gives much better developer

ergonomics, especially for managing inter-task communication and for handling interaction with the hardware.

3.2 Defmt

The second piece of the system is *defmt* [1]: it is a logging framework for embedded applications, written in Rust.

It uses a novel approach to improve space and computational efficiency of the logging on the embedded device side. Instead of storing the format strings and perform parameter formatting on the device, it *defers* those operations, moving them on the connected host.

Technically, it embeds metadata about format string and the parameters types on a linker section that gets stripped before placing the firmware into the device flash. Thus, the device only needs to send the format string index and raw parameters; then the connected host, provided with the non-stripped binary file, will perform formatting and printing.

The disadvantage of this approach is that without the metadata embedded in the binary file, the messages from the device are not interpretable. Nonetheless, the advantages in terms of size and computational requirements outweighs the downsides in most cases.

Defmt supports different *backends* for communication between the host and the device, such as *RTT*, *semihosting* and *ITM*; in our implementation we opted to use *RTT*, as we had problems with the semihosting backend.

3.3 HAL

The final key component of the system is the *Hardware Abstraction Layer (HAL)*. It provides a high-level, portable interface to interact with the underlying hardware peripherals, abstracting away the low-level register manipulation and platform-specific details.

In the Rust embedded ecosystem, *HAL*s are typically implemented as crates that follow the *embedded-hal* trait specifications [6]. This standardization allows for interoperability between different hardware platforms and peripheral drivers, enabling code reuse and ecosystem compatibility.

For our implementation, we used the *STM32F4xx HAL* crate [7], which provides abstractions for *GPIO* pins, timers, *UART* communication, *SPI*, *I2C*, and other peripherals specific to the *STM32F4* microcontroller family. The *HAL* ensures type-safe access to hardware resources at compile time, preventing common errors such as accessing uninitialized peripherals or conflicting resource usage, enforced by Rust strong type system. In general *RTIC* is *HAL* agnostic: it is possible to use other implementations from different vendors (e.g. *Embassy*).

The most important peripheral needed for the experiment is the timer, which is used to provide the notion of time to the system, via a global *rtic-monotonics* object, thus enabling time-based operations (e.g. delays, periodic tasks, deadline monitoring). *RTIC* has built-in support for our chosen *HAL*, hence implementing the required timer object only required calling a simple macro.

```
    pub trait Profiler {
      fn reset(&mut self);
      fn lap(&mut self);
      fn lap_time(&self, lap: usize) ->
      ↪  Option<ProfilingDuration>;
      fn update_wcet(&mut self);
      fn log(&self);
}
```

Listing 1: Profiler trait used to extract WCET

4 Execution Times

4.1 Rust

Obtaining the *Worst-Case Execution Times* for every operation in the system is a crucial step in order to perform response-time analysis.

We decided to leverage the *Data Watchpoint and Trace Unit* (DWT) [8, Chapter 9]. DWT supports multiple functionalities such as: debugger hardware watchpoints, multiple comparators, and cycle counters. For our purposes, the latter one was sufficient.

In particular, the DWT peripheral holds a cycle counter that can be read from the `DWT_CYCCNT` register, which in turn can be used to implement a simple *stopwatch* that can measure time spans.

The chosen HAL [7] already provides a stopwatch implementation that correctly handles the wrap around of the clock counter. The typical system clock frequency of our board is 168 MHz, which makes the period $T = \frac{1}{f} \simeq 5.95\,\text{ns}$. Since the counter size is 32 bits, it will wrap around every $2^{32} \cdot T \simeq 25\,\text{s}$, thus it must be handled to prevent wrong readings.

The stopwatch implementation provides an API that closely resembles the workings of a manual stopwatch: it exports a `reset` function that simply clears every stored time, and a `lap` method that stores the current cycle count into a buffer passed during initialization. Lap times can be simply obtained by subtracting successive cycle counts, having care of handling wrap around.

Since this implementation is really basic, we have designed a higher-level abstraction that allows us to obtain the WCET of selected code sections, corresponding to the operations under analysis. Listing 1 shows the *trait* definition for our profiler abstraction. The `lap` method gets called at the end of every iteration (i.e. job execution), using the saved laps; then results can be logged using the corresponding method.

Every different task gets a tailored implementation of the profiler *trait*; they mainly differ on the implementation of the `update_wcet` method, to accommodate the different operations inside the tasks.

The profiling instrumentation is gated behind `cargo` *features*: when the user desires to start the profiling of a certain operation it needs to enable the related feature at compile-time. This ensures that the application is completely devoid of profiling objects and instructions that are not needed during normal operation, improving performance and code size.

The runtime does not provide a *task time* like Ada Ravenscar, hence, to obtain the desired WCET, we had to manually raise the priority of the task under analysis in order to prevent any interference that could invalidate our times. This is not a very clean solution, but implementing a proper task execution time would

require non-trivial modifications to the RTIC runtime, which were considered out of the scope of the work and too time-consuming. Nonetheless, the WCET obtained remain valid.

We have decided to take the maximum execution time among one-hundred executions; when the correct profiling feature is enabled, the tasks are configured to automatically log the WCET results accumulated by the profiler and then panic to stop execution.

4.2 Ada

For the Ada system we have re-utilized the instrumentation implemented by J. Hu and A. Gobbo in their work [13], aimed to perform response-time analysis over the same system model shown in the original paper [10].

4.3 WCET Times

This section contains the obtained WCET values. Both implementations have been compiled in *release* mode (i.e. with all optimizations enabled), and run on the board at maximum clock speed (168 MHz).

We have observed some significant differences in execution times for certain operations between the Ada and Rust system. We ascribe these gaps to differences in I/O backends and runtime overheads, since RTIC heavily relies on hardware features specific to ARM Cortex architecture, to compiler choices, and different math libraries.

Operation	Rust (ns)	Ada (ns)
Small Whetstone (workload 139)	4346391	1018910
AL Read	136	1267
ALR Read	647	3256
Put Line	8017	8222

Table 3: WCET per operation — Activation Log Reader

4.3.1 *Activation Log Reader.*

Operation	Rust (ns)	Ada (ns)
AL Write	583	1678
EES Write	1071	3322

Table 4: WCET per operation — External Event Server

4.3.2 *External Event Server.*

Operation	Rust (ns)	Ada (ns)
RB Extract	303	1361
Extract Workload	814	4760
Small Whetstone (workload 278)	8692782	2037820
Put Line	7970	7406

Table 5: WCET per operation — On-Call Producer

4.3.3 *On-Call Producer.*

Operation	Rust (ns)	Ada (ns)
Small Whetstone (workload 756)	23639364	5541700
Due Activation	214	1344
RB Deposit	2398	3583
OCP Activation	2903	5283
Check Due	208	1372
ALR Signal	1982	6922
Cancel Deadline	273	–
RP Cancel Deadline	8314	–

Table 6: WCET per operation — Regular Producer

4.3.4 *Regular Producer.*

Operation	Rust (ns)	Ada (ns)
Systick (worst case)	702	–
Systick Overhead	32	–
ISR Switch	2732	–
Delay Until	865	–
Signal RTIC	4160	–
Task Semaphore	4178	–
Event Queue	4101	–
Spawn Overhead	148	–
Context-Switch	166	–

Table 7: WCET — Kernel/Runtime Overheads

4.3.5 *Kernel/Runtime Overheads.*

5 Deadline Miss Detection

In our system, each task is associated with an hard deadline, as described in Table 1. In order to detect a deadline miss, since the *RTIC* framework does not provide any built-in mechanism for this, for each task we implemented a watchdog task responsible for monitoring its deadline.

To achieve the desired behaviour, i.e., a watchdog declares a deadline miss if and only if the associated task fails to complete its execution before its deadline, we used a shared resource, called `deadline_protected_object`. This resource is a struct containing a boolean flag, which value rappresents whether the task has completed its execution or not. In *RTIC* framework, each shared resource needs to be acquired before being accessed or before invoking any of its associated methods. Therefore, when a monitored task acquired the lock on the `deadline_protected_object` to invoke `cancel_deadline`, the method responsible for acknowledging the deadline, it can not be preempeted by its watchdog task. This prevents the watchdog from preempting the monitored task while it is updating the flag, avoiding undesired deadline miss detections.

Additionaly, the `deadline_protected_object` struct contains an activations counter, which allows for finer control over the requests to clear deadline misses, allowing stale requests to be ignored.

Since in our system we have periodic and sporadic tasks, we implemented two different types of watchdogs:

- `periodic_deadline_watchdog`: this watchdog has as locals are the period and the next deadline of the monitored task. It delays itself until the next deadline and then, after acquiring the shared `deadline_protected_object`, it invokes on it the method `deadline_miss_detected`. This method checks the flag and, if it is still false, it declares a deadline miss. Then, the watchdog increments the next deadline by the period of the monitored task and it loops again;
- `sporadic_deadline_watchdog`: this watchdog waits on an activation signal that notices it of a new activation of the monitored task. The watchdog then calculates the next deadline by adding the relative deadline with the activation time of the monitored task, passed as argument inside the precedent signal. The watchdog then delays itself until the obtained absolute deadline and, after acquiring the shared `deadline_protected_object`, it invokes on it the method `deadline_miss_detected`, as happens in the `periodic_deadline_watchdog` case. After that, it loops again, waiting for the next activation signal.

To avoid the first activation of a periodic watchdog from starting before the monitored task can actually execute due to system initialisation, we set its first deadline equal to the system activation instant plus the task's relative deadline. The system activation instant consists in a static `AtomicU32` rappresenting, in ticks, the instant befoore which all initializtion-spawned tasks are delayed: this mechanisms provides the system with a well defined "*zero time*".
Moreover, in order to provide accurate deadline management for sporadic tasks, their activation signals occur inside a critical section. This prevents unwanted preemptions between the task activation and the sending of the signal to the corresponding watchdog, which could otherwise lead to an incorrect calculation of the next deadline.

6 MAST

The response time analysis of the system has been largely performed using the *MAST* tool. *MAST* is a *Modeling and Analysis Suite for Real-Time* applications, developed at the *University of Canatabria*. It enables the modelling of real-time systems and provides a set of tools for performing various types of analysis on a given model.
In particular, for the purpose of this project, its ability to perform response-time analysis for real-time systems on a single-processor using *Fixed-Priority Scheduling* policy. The tool also supports preemptive tasks and the *Immediate Ceiling Protocol* for managing shared resources.

6.1 MAST Model

To analyze the system using *MAST*, a model of the system must be provided in the form of an XML or text file. In this sub-section, we describe the model components available for representing the system. Furtrher details can be found in the *MAST description*.

6.1.1 Processing_Resource. It represents a resource capable of executing abstract activities. In our case, since we are dealing with a single-processor system, there is only one processing resource. When we define a processing resource, we must provide the range of priorities available, a *System_Timer* and the worst *ISR Switch* time. The latter parameter denotes the context switch time of an interrupt service routine, without taking into account the execution time of the ISR itself.

6.1.2 System_Timer. It represents a timer used by a processing resource. In our system we have to deal with *Ticker*, a system that has a periodic ticker. The informations required by this type of timer are the period and the worst-case overhead time, i.e. the time taken to process the ticker interrupt, including the scheduling overhead required for scanning and moving the jobs in the ready queue.

6.1.3 Scheduler. It represents the scheduling policy used to manage the amount of CPU processing capacity. As previously mentioned, our system uses a *Fixed-Priority Scheduling* policy. When defining a scheduler, we need to specify the processing resource it is hosted on, the available priority range and the worst context switch time. This last parameter denotes the time taken to set the context switch interrupt as pending and to execute its handler, which is responsible for saving registers of active context and for restoring the ones of the new one.

6.1.4 Shared_Resource. It represents a resource that is shared among tasks. As stated before, the *MAST* model uses the *Immediate Ceiling Protocol*; therefore the priority specified for a shared resource is its ceiling priority, i.e. the highest priority of the tasks that access it.

6.1.5 Scheduling_Server. It represents a schedulable entity in a processing resource. Since we use a *Fixed-Priority Scheduling* policy, the server scheduling parameters are of type *Fixed_Priority_Policy*, which corresponds to regular preemptive fixed-priority scheduling, and require a priority value to be specified.

6.1.6 Operation. It is an abstract representation of a piece of code. There are three types of operations:

- *Simple*: used to model simple operations or methods of protected objects. In the latter case, the operation requires a list of shared resources to lock before executing it and to unlock after its completion. It also requires the specification of best, average and worst-case execution times;
- *Composite*: an operation composed of an ordered sequence of other operations. Its execution times are derived from the execution times of its internal operations;
- *Enclosing*: similar to a composite one but we need to specify the best, average and worst-case execution times as well. This provides a way to express overheads associated with inner operations, such as the cost of acquiring and releasing a shared resource.

6.1.7 Transaction. It represents a modeling unit for interactions of the system. Each transaction is composed of three components:

- *External_Events*: a list of events that models interactions of the system with external devices through interrupts or with hardware timing devices. For each external event it is necessary to specify its type, such as periodic or sporadic, its period or minimum inter-arrival time, its jitter and its phase;
- *Internal_Events*: a list of events that are generated by event handlers. They can be used to express deadlines through *Timing_Requirements*: in our case, we use *Hard_Global_Deadline*

timing requirements, which specify an hard deadline that must be met by all instances of the internal event, and a reference to the external event that represents its activation;
- *Event_Handlers*: a list of event handlers that enable to express relationships between events. Each event handler requires an event (or a list of events) as input and produces an event (or a list of events) as output. Based on the type of the event handler (such as *System_Timed_Activity*), it is also possible to execute an activity operation on a specified activity server.

6.2 MAST Tools

MAST supports different types of analysis tools. In this subsection, we present those used in our system's analysis, while further details can be found in the *MAST description*.

- *Classic RM Analysis*: the classic exact response-time analysis for single-processor fixed-priority systems. Although the name might suggest support only for *Rate Monotonic* policy, it also covers *Deadline Monotonic* as well, which is the policy adopted in our system;
- *Offeset-based Approximate with Precedence Relation Analysis*: response-time analysis that improves the holistic analysis by taking into account that tasks of the same transaction are not independent, through the use of offsets. Those precedence relations, togheter with tasks priorities, provide a tighter estimation of the response times comparing to other tools.

Each analysis tool, as described in the *MAST restrictions*, is subject to certain limitations regarding the expression power of the model, particulary in how transactions can be constructed. The most relavant are:

- *Restricted_Multipath_Transactions*: it imposes the absence of branch elements in side transactions;
- *Referenced_Events_Are_External_Only*: it requires only external events as referencable events, therefore only external events can be used as activation events for deadlines;
- *Linear_Transactions_Only*: it allows just one external event per transaction.

7 FPS Analysis

In this section we present our *Fixed-Priority Scheduling (FPS)* analysis of the system. The analysis was performed by modeling the system through a *MAST* model, which was then analyzed using the *MAST* tool to verify the timing properties and to analyze the system under different workloads.
The priority assignment was done according to the *Deadline Monotonic* policy, which assigns higher priorities to tasks with shorter deadlines. This choice comes from the fact that tasks in the system do not have implicit deadlines and, as theory proves, *Deadline Monotonic* performs better than *Rate Monotonic* in such cases, as it can provide feasible schedules where *Rate Monotonic* fails.
The evaluation of the system under different utilization levels was facilitated by the use of the *Small Whetstone* algorithm to simulate computational workloads. Thanks to its configurable load and deterministic behaviour without involving the use of shared resources, it provides a straightforward way to test the system under differen utilizations by simply scaling the WCET in the different `small_whetstone` operations that are present in the *MAST* model, without the need for new execution-time measurements.

7.1 Holistic Analysis

We first conducted an holistic analysis, considering all the tasks as intendepent. As a consequence, this analysis is pessimistic, since it does not take into account the relationships between tasks. System feasibility is therefore evaluated based on the worst-case scenario at the critical instant, i.e., when all tasks are released simultaneously (not a realistic and possible case). The analysis was performed using the *Classic RM Analysis* tool in *MAST*, which implements the classic exact response-time analysis for single-processor systems under *Fixed-Priority Scheduling* (although it's called *Rate Monotonic*, it bases *Deadline Monotonic* analysis).
The first results of the analysis were made considering the system under the loads provided in the original *Guided Extended Example*, i.e., with the following Whetstone workloads (accompained by their corresponding WCETs expressed in seconds).

Task	Workload	Rust WCET	Ada WCET
`Regular_Producer`	756	0.023639364	0.00554170
`On_Call_Producer`	278	0.008692782	0.00203782
`Activation_Log_Reader`	139	0.004346391	0.00101891

Table 8: Whetstone Operations

The obtained results are reported in Table [9] and Table [13]. Times are expressed in seconds and we omitted the watchdogs' transactions (which were instead included in the Rust model) since they are not relevant for identifying increasing workload factors. Regarding the jitter times, all transactions suffer jitter due to the system ticker interrupt and the context switch overhead. Moreover, additional sources of jitter and blocking times for each transaction are the following:

- `rp_transaction`: suffers additional jitter due to the possible interference of the `ees_transaction`. Its blocking time is instead due to the possible access to shared resources from the lower-priority `ocp_transaction` and `alr_transaction` with ceiling priority equal to the priority of `rp_transaction`;
- `ocp_transaction`: suffers additional jitter due to interference by the higher-priority `rp_transaction`. Its blocking time is caused by the `alr_transaction`;
- `alr_transaction`: suffers additional jitter due to interference by both the higher-priority `rp_transaction` and `ocp_transaction`. It does not suffer blocking, since it represents the lowest-priority task in the system;
- `ees_transaction`: it suffers no additional jitter than the aforementioned overheads. Its blockig time is caused by the `alr_transaction` which can possibly access the shared `Activation_Log` resource with ceiling priority equal to the priority of `ees_transaction`.

Transaction	R_{max}	Slack	Jitter	MBT
rp_transaction	0.024728	2006.3%	0.001066	3.030E-06
ocp_transaction	0.033451	8791.0%	0.024741	2.730E-06
alr_transaction	0.037822	21927.3%	0.033455	0.000
ees_transaction	0.001043	>=100000.0%	2.955E-05	2.730E-06
System Utilization	2.79%			
System Slack	1994.1%			

Table 9: Holistic Analysis Results Rust

Based on these results, since the smallest slack available was 2006.3% for the rp_transaction, we increased the workloads with a factor of 20 to reduce it significantly. New results are reported in Table [10].

Transaction	R_{max}	Slack	Jitter	MBT
rp_transaction	0.474326	5.08%	0.001516	3.030E-06
ocp_transaction	0.648377	87.11%	0.474643	2.730E-06
alr_transaction	0.735413	303.91%	0.787757	0.000
ees_transaction	0.001043	>=100000.0%	0.001030	2.730E-06
System Utilization	53.76%			
System Slack	5.75%			

Table 10: Holistic Analysis Results Rust x20

We can notice how the blocking times have not changed, since protected operations remain the same. While we have achieved a small slack for the rp_transaction (5.08%), the other two transactions that involve Whetstone computations still have large available slacks. Based on these results, especially on the smallest slack of 87.11%, we decided to increase the On_Call_Producer and Activation_Log_Reader workloads by a factor 1.8, while keeping the Regular_Producer workload fixed. The updated results are reported in Table [11].

Transaction	R_{max}	Slack	Jitter	MBT
rp_transaction	0.474326	2.34%	0.001516	3.030E-06
ocp_transaction	0.787600	3.91%	0.473643	2.730E-06
alr_transaction	0.944248	35.55%	0.786757	0.000
ees_transaction	0.001043	91855.5%	0.001030	2.730E-06
System Utilization	58.86%			
System Slack	1.58%			

Table 11: Holistic Analysis Results Rust x1.8

Finally, based on the last useful available slack of 35.55% for the alr_transaction, we increased its workload by a factor of 1.35. The updated results are reported in Table [12].

Transaction	R_{max}	Slack	Jitter	MBT
rp_transaction	0.474326	0.00%	0.001516	3.030E-06
ocp_transaction	0.787600	0.00%	0.474643	2.730E-06
alr_transaction	0.999068	0.39%	0.787812	0.000
ees_transaction	0.001043	6911.7%	0.001030	2.730E-06
System Utilization	60.68%			
System Slack	0.39%			

Table 12: Holistic Analysis Results Rust x1.35

Even though the ees_transaction has still a large available slack, since it does not perform any Whetstone computation, it is not possible to increase the utilization of the system any further. Besides that, even an eventual increase of the execution time of ees_transaction would not affect significantly the utilization of the system. In conclusion, under an holistic analysis, we achieved an utilization of the system equal to 60.68%.

Transaction	R_{max}	Slack	Jitter	MBT
rp_transaction	0.006624	8808.2%	0.001045	1.361E-06
ocp_transaction	0.007697	38006.3%	0.005638	1.267E-06
alr_transaction	0.008752	91394.9%	0.007706	0.000
ees_transaction	1.882E-05	>=100000.0%	1.141E-05	1.267E-06
System Utilization	1.03%			
System Slack	7728.7%			

Table 13: Holistic Analysis Results Ada

Based on these results, since the smallest available slack was 8808.2% for the rp_transaction, we increased the workloads with a factor of 90 to reduce it significantly. The updated results are reported in Table [14].

Transaction	R_{max}	Slack	Jitter	MBT
rp_transaction	0.490670	1.56%	0.002963	1.361E-06
ocp_transaction	0.669740	72.27%	0.490390	1.267E-06
alr_transaction	0.759792	265.23%	0.670102	0.000
ees_transaction	1.882E-05	>=100000.0%	1.141E-05	1.267E-06
System Utilization	55.74%			
System Slack	1.98%			

Table 14: Holistic Analysis Results Ada x88

Again, we can notice how the blocking times have not changed, since protected operations remain the same. While we have achieved a small slack for the rp_transaction (1.56%), the other two transactions that involve Whetstone computations still have large available slacks. Based on these results, especially on the smallest slack of 72.27%, we decided to increase the On_Call_Producer and Activation_Log_Reader workloads by a factor of 1.7, while keeping the Regular_Producer workload fixed. The updated results are reported in Table [15].

Transaction	R_{max}	Slack	Jitter	MBT
rp_transaction	0.490670	0.78%	0.002963	1.361E-06
ocp_transaction	0.795769	1.17%	0.490889	0.000
alr_transaction	0.948836	32.42%	0.796380	1.361E-06
ees_transaction	1.882E-05	56921.9%	1.141E-05	1.267E-06
System Utilization	60.35%			
System Slack	0.78%			

Table 15: Holistic Analysis Results Ada x1.7

Finally, based on the last useful slack available of 32.34% for the alr_transaction, we increased its workload by a factor of 1.3. The updated results are reported in Table [16].

Transaction	R_{max}	Slack	Jitter	MBT
rp_transaction	0.490670	0.78%	0.002963	1.361E-06
ocp_transaction	0.795769	1.17%	0.490889	0.000
alr_transaction	0.994747	1.95%	0.796563	1.361E-06
ees_transaction	1.882E-05	56921.9%	1.141E-05	1.267E-06
System Utilization	61.87%			
System Slack	0.78%			

Table 16: Holistic Analysis Results Ada x1.3

Again, the considerations made before upon the ees_transaction apply here. In conclusion, under an holistic analysis, we achieved an utilization of the system equal to 61.87%.

7.2 Offset-Based Analysis

To improve the results obtained with the holistic analysis, we performed an offset-based analysis, which takes into account the relationships between tasks, thus providing less pessimistic results.
To exploit the benefits of offsets, we use the *Offset-Based Analysis with Precedence Relation* tool provided in *MAST*. Due to restrictions imposed by this model, we had to represents each periodic regular_producer activation as a separate transaction, plus the additional transaction required for the external_event_server. The on_call_producer and activation_log_reader activations are included within the rp_transactions when needed, exploiting the system-imposed order, i.e., the activation_log_reader always executes after the on_call_producer, which, in turn, always executes after the regular_producer (if they are activated in the same period). Since the hyperperiod of the system is 15 seconds, we have 15 rp_transactions plus the ees_transaction, resulting in a total of 16 transactions.

If the offset-based analysis helps in reducing the interference between tasks, we increase increase the workloads beyond what was achieved with the holistic analysis. The reason is that the critical instant faced in the holistic analyis is similar to the one faced with the offset-based analysis in the 12-th period, when the regualr_producer activates both the on_call_producer and the activation_log_reader. A graphical representation of this scenario is shown in Figure [2], where we omitted the inclusion of ees_transaction.

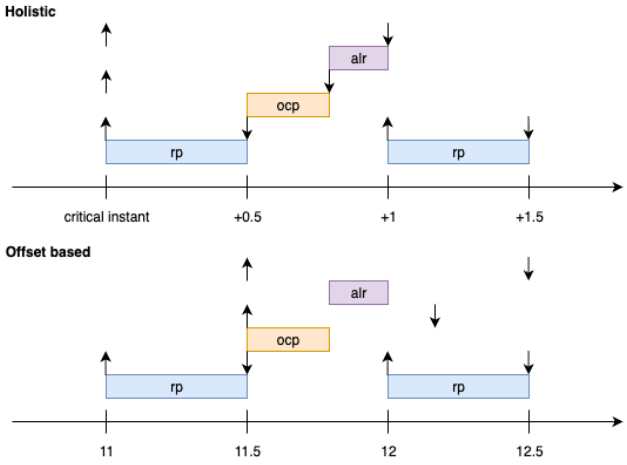

Figure 2: Holistic Analysis vs Offset-Based Analysis at the 12-th Period

The figure shows how the workloads obtained with the holistic anlysis already fill the available slacks also in the offset-based analysis.
To further increase the utilization of the system, we decided to take advantage of those tasks which activate more frequently during the hyperperiod. While the regular_producer, which activates every second, already executes for almost all of its executable period time, until its deadline, we noticed how the activation_log_reader, which has a period of 3 seconds, has a workload that is lower than the one of the regular_producer, even though it has a period of 5 seconds. This implies that, during the hyperperiod, the activation_log_reader activates 5 times, whereas the on_call_producer activates only 3 times. Of these activations, only in the 12th period do both tasks execute in the same period. Moreover, since the regular_producer has a period of 1 second and triggers both the on_call_producer and activation_log_reader, and considering that it has a higher priority than both, the available execution time for these two tasks is limited to 0.5 seconds, even though their respective deadlines (0.8 and 1 second) would allow for longer execution.

Since our goal is to maximize the total utilization, we need to increase as much as possible to fill the identified 3.5 seconds (there are 8 intervals due to 8 total activations, but since one is shared, we obtain 7 intervals of 0.5 seconds each).
Before expressing the situation as an optimization problem, several considerartions must be taken into account. First, we need to account for the system ticker overhead, which can be estimated as:

$$\text{Tot Sys Ticker Overhead} = \frac{0.500}{0.001} * \text{sys ticker worst overhead}$$

Next, we must consider the dispatching cost that occurs when the on_call_producer completes and the activation_log_reader begins its execution. This overhead is pessimistically estimated as the worst-case context switch time.
Finally, we need to consider the possible execution of the ees_transaction: since the external_event_server has an higher priority than both the tasks (this is not true for the

`activation_log_reader` if it had acquire the `activation_log`, thanks to the `Stack Resource Protocol`), we must account for the cost of preemption (i.e., the context switch) plus the WCET of `ees_transaction`.
We ends up with the following available times for the two systems:

- Rust System: 0.5 - 0.0005 - 0.018E-06 - (0.018E-06 + 13.486E-06) = 0.49948648 seconds;
- Ada System: 0.5 - 0.001928 - 3.964E-06 - (3.964E-06 + 4.303E-06) = 0.49805977 seconds.

The problem can now be expressed as the following linear optimization problem:

Let x = execution time of `on_call_producer`
y = execution time of `activation_log_reader`
T = available time: 0.49948648 s (Rust), 0.49805977 s (Ada)

Maximize $3x + 5y$

Subject to $x \leq \mathrm{T}$
$y \leq \mathrm{T}$
$x + y \leq \mathrm{T}$
$x, y \geq 0$

For both the systems, the solution to this linear optimization is $x = 0$ and $y = T$, meaning that the `activation_log_reader` task should occupy the entire available time slot. Since this is an unrealistic scenario that goes againt the meaning of the system, we chose to assign the minimum workload possible to the `on_call_producer`, i.e., a workload equal to a single Whetstone operation. The remaining time in T is then assigned to the `activation_log_reader`, resulting in 0.49948648 - 0.00003127 = 0.49945521 seconds for the Rust system and 0.49805977 - 0.0000073303 = 0.49805244 seconds for the Ada system.

With these updated workloads, the offset-based model, with the *Offset-Based Analysis with Precedence Relation* tool in *MAST*, yields the following results:

- Rust Total Utilization: 64.03%;
- Ada Total Utilization: 65.77%.

Since the transactions have the same activation source (a `regular_producer` job), there is no interference between tasks. Therefore, to validate our assumptions, we just consider only wheter best-case response times are within the deadlines, ignoring the worst-case response times as well as slacks.

8 Conclusions

In this technical report, we presented the implementation of a real-time system in Rust and we compared it with an equivalent implementation in Ada. The analysis, conducted with the *MAST* tool, showed that both implementations are able to achieve similar results in terms of system utilization. To support the analysis, both systems were also tested on an *Olimex STM32H405* [15] using the identified workloads, and no missed deadlines were observed.
Rust, and in particular the *RTIC* framework, proved to be a valid alternative for the development of real-time systems.
Regarding the *RTIC* framework, by being a declarative system model for single-core applications, it enforces a well-defined structure, requiring a clear understanding and strict control over the management of both local and shared resources. This strong structure can support developers in maintaining a well-organized and systematic design of the system. However, in some situations it may also limit design flexibility in relation to *Ada Ravenscar Profile*, especially when dealing with shared resources. In particular, we were unable to separate the responsibility of the `on_call_producer` activation, as it was not possible to implement it except by embedding it directly within the `deposit` call on the `Request_Buffer`. This limitation stems from the fact that any access to a shared resource must be explicitly declared in the task's shared resources and, consequently, it is not possible for a task to provide an interface through which other tasks may interact with it. The management of shared resources always requires acquiring a lock when accessing them, which can introduce unnecessary blocking times even in cases where the relationships between tasks exclude the possibility of simultaneous access. Once again, this enforced structure represents the trade-off between flexibility and the strong safety guarantees provided by Rust.
Unlike the *Ada Ravensacar Profile*, on the subject of deadline miss detenctions, the framework reveals a significant limitation by providing no built-in support for handling such events, thus it leaves the responsability for handling timing violations entirely to the developers.
Finally, we would like to highlight that the *RTIC* framework is still a valid option for the development of real-time systems, with a generated code that, as shown in this report, infers no substantial overhead compared to an equivalent implementation in *Ada Ravenscar Profile*.

9 Citations and Bibliographies

References

[1] [n. d.]. Introduction - Defmt Book. https://defmt.ferrous-systems.com/.
[2] [n. d.]. MAST: A Comprehensive Tool Suite for Real-Time Systems Analysis and Optimization – ACM SIGBED.
[3] RTIC - The hardware accelerated Rust RTOS [n. d.]. *RTIC*. RTIC - The hardware accelerated Rust RTOS. https://rtic.rs/2/book/en/
[4] [n. d.]. ST-LINK/V2 | Tool - STMicroelectronics. https://www.st.com/en/development-tools/st-link-v2.html.
[5] 2025. Rtic-Rs/Rtic. Real-Time Interrupt-driven Concurrency (RTIC).
[6] 2025. Rust-Embedded/Embedded-Hal. Rust Embedded.
[7] 2025. Stm32-Rs/Stm32f4xx-Hal. stm32-rs.
[8] ARM Limited. 2009. ARM Cortex M4 Technical Reference Manual.
[9] T. P. Baker. 1991. Stack-Based Scheduling of Realtime Processes. *Real-Time Systems* 3, 1 (March 1991), 67–99. doi:10.1007/BF00365393
[10] Alan Burns, Brian Dobbing, and Tullio Vardanega. 2004. Guide for the Use of the Ada Ravenscar Profile in High Integrity Systems. *Ada Lett.* XXIV, 2 (June 2004), 1–74. doi:10.1145/997119.997120
[11] Albert M. K. Cheng and James Ras. 2007. The Implementation of the Priority Ceiling Protocol in Ada-2005. *Ada Lett.* XXVII, 1 (April 2007), 24–39. doi:10.1145/1274610.1274611
[12] H. J. Curnow and B. A. Wichmann. 1976. A Synthetic Benchmark. *Comput. J.* 19, 1 (Jan. 1976), 43–49. doi:10.1093/comjnl/19.1.43
[13] Jiayi Hu. 2024. Jiayihu/SRT.
[14] Jane W. S. Liu. 2000. *Real-Time Systems.* Prentice Hall, Upper Saddle River, NJ.
[15] Olimex. [n. d.]. STM32-H405. https://www.olimex.com/Products/ARM/ST/STM32-H405/.
[16] L. Sha, R. Rajkumar, and J.P. Lehoczky. 1990. Priority Inheritance Protocols: An Approach to Real-Time Synchronization. *IEEE Trans. Comput.* 39, 9 (Sept. 1990), 1175–1185. doi:10.1109/12.57058